

Laporan Praktikum Kontrol Cerdas

Nama : Adiesceasha akbar

NIM : 224308001

Kelas : TKA-6A

Akun Github (Tautan) : <https://github.com/adiesakbar>

Student Lab Assistant : Muhammad Fatih Anfa Ahima (214308092)

1. Judul Percobaan

Machine Learning for Control Systems

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

Pada akhir praktikum ini, mahasiswa akan dapat memahami dasar-dasar Machine Learning dalam sistem kendali. Mengimplementasikan model ML sederhana untuk klasifikasi objek. Menggunakan Scikit-learn untuk membuat model ML dasar. Mengintegrasikan model ML dengan Computer Vision untuk deteksi objek. Mengelola dataset dan melakukan pelatihan model sederhana.

3. Landasan Teori

Peran Machine Learning dalam Sistem Kendali Machine Learning (ML) memungkinkan sistem kendali untuk belajar dari data dan meningkatkan performanya seiring waktu. Beberapa contoh penerapannya: ✓ Predictive Maintenance: Mendeteksi potensi kegagalan sistem sebelum terjadi. ✓ Adaptive Control: Sistem kendali yang dapat menyesuaikan parameter secara otomatis. ✓ Anomaly Detection: Mendeteksi perilaku tidak normal dalam sistem industri. Konsep Dasar Machine Learning ML terdiri dari beberapa pendekatan utama: 1. Supervised Learning: Model belajar dari data berlabel (contoh: klasifikasi objek berdasarkan warna). 2. Unsupervised Learning: Model belajar dari pola dalam data tanpa label (contoh: clustering objek berdasarkan fitur visual). 3. Reinforcement Learning: Model belajar melalui trial-and-error untuk mengoptimalkan keputusan.

4. Analisis dan Diskusi

ANALISIS:

1. Bagaimana kinerja model dalam mendeteksi warna?

Keakuratan model dalam mengenali warna bergantung pada berbagai faktor, termasuk kualitas dataset, jumlah sampel, pemilihan parameter K dalam KNN, serta metode ekstraksi fitur yang diterapkan. Jika dataset memiliki variasi warna dan pencahayaan yang cukup, model dapat mengidentifikasi warna dengan tingkat akurasi yang tinggi. Namun, jika terdapat warna yang sangat mirip, model mungkin mengalami kesulitan dalam membedakannya.

2. Bagaimana pengaruh penambahan jumlah dataset terhadap akurasi?

Secara umum, menambah jumlah dataset dapat meningkatkan akurasi model karena lebih banyak sampel memungkinkan model memahami pola warna dengan lebih baik. Namun, jika data tambahan mengandung banyak noise atau distribusinya tidak merata, justru dapat menyebabkan penurunan akurasi.

3. Bagaimana cara meningkatkan kinerja model klasifikasi?

Optimasi parameter K → Menentukan nilai K yang optimal menggunakan metode seperti cross-validation.
Normalisasi data → Memastikan fitur warna telah dinormalisasi agar perhitungan jarak dalam KNN lebih akurat.

Penambahan dataset → Meningkatkan jumlah sampel dengan variasi warna yang lebih luas untuk mengurangi bias model.

Feature engineering → Menggunakan representasi warna yang lebih informatif, seperti HSV dibandingkan dengan RGB.

Menerapkan metode lain → Jika KNN kurang efektif, dapat mempertimbangkan algoritma lain seperti SVM atau CNN.

DISKUSI :

1. Apa keunggulan Machine Learning dibandingkan metode berbasis aturan (rule-based)?

Lebih fleksibel → ML dapat mempelajari pola dari data tanpa perlu aturan manual yang kompleks.
Mampu mengenali pola kompleks → ML dapat mendeteksi hubungan yang sulit didefinisikan dengan aturan eksplisit.
Mudah beradaptasi → Model ML dapat diperbarui dengan data baru tanpa perlu melakukan perubahan kode secara menyeluruh.

2. Bagaimana integrasi Machine Learning dalam sistem kendali?

Otomatisasi deteksi warna dalam robotika → Robot dapat mengenali objek berdasarkan warna secara otomatis.
Pengendalian kualitas di industri manufaktur → ML dapat digunakan untuk mendeteksi cacat warna pada produk.
Sistem pengenalan warna dalam visi komputer → Diterapkan pada kendaraan otonom atau sistem pengawasan berbasis visual.

3. Apa tantangan dalam penerapan Machine Learning pada sistem real-time?

Latensi tinggi → Prediksi harus dilakukan dengan cepat agar dapat digunakan secara langsung.
Perubahan kondisi pencahayaan → Warna dapat tampak berbeda tergantung pencahayaan, sehingga diperlukan teknik preprocessing yang baik.
Keterbatasan perangkat keras → Model ML yang kompleks membutuhkan daya komputasi tinggi.
Risiko overfitting → Model dapat berkinerja baik pada data pelatihan tetapi kurang optimal dalam situasi nyata.

5. Assignment

Pada tugas ini, mahasiswa modifikasi program untuk menggunakan model ML lain (contoh: Decision Tree atau SVM). Tambahkan fitur untuk mendeteksi lebih dari satu warna secara simultan. Eksplorasi dataset lain di Kaggle dan coba implementasikan model pada dataset tersebut.

6. Data dan Output Hasil Pengamatan

No	Variabel	Hasil data
1	knn	<pre>10 # multi color sample 11 # break 12 13 height, width, _ = frame.shape 14 15 # tentukan beberapa titik sampel dari 16 sample_points = 1 17 (height // 2, width // 2), # 1 18 (height // 4, width // 4), # 2 19 (height // 4, 3 * width // 4), # 3 20 (3 * height // 4, width // 4), 21 (3 * height // 4, 3 * width // 4) 22] 23 24 colors_detected = [] 25 for (y, x) in sample_points: 26 pixel = frame[y, x] 27 pixel_scaled = scaler.transform 28 color_pred = svm_model.predict 29 colors_detected.append((x, y, c 30 31 # Tampilkan warna yang terdeteksi di 32 for (x, y, color) in colors_detected: 33 cv.putText(frame, f'color: {color}', (x, y), cv.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 0), 2)</pre>

2	Svm	<pre>1 import cv2 2 import joblib 3 import numpy as np 4 5 # Must model svm serta scaler 6 svm_model = joblib.load('data svm.pkl') 7 scaler = joblib.load('scaler.pkl') 8 9 # inialisasi kamera 10 cap = cv2.VideoCapture(0) 11 12 while True: 13 ret, frame = cap.read() 14 if not ret: 15 break 16 17 #ambil pixel tengah gambar 18 height, width, _ = frame.shape 19 pixel_center = frame[height//2, width//2] 20 21 # Normalisasi pixel sebelum prediksi 22 pixel_center_scaled = scaler.transform([pixel_center])</pre> SVM: Cool Grey 
3	Multi color	<pre>1 import cv2 2 import joblib 3 import numpy as np 4 5 # Must model svm serta scaler 6 svm_model = joblib.load('data svm.pkl') 7 scaler = joblib.load('scaler.pkl') 8 9 # inialisasi kamera 10 cap = cv2.VideoCapture(0) 11 12 while True: 13 ret, frame = cap.read() 14 if not ret: 15 break 16 17 #ambil pixel tengah gambar 18 height, width, _ = frame.shape 19 pixel_center = frame[height//2, width//2] 20 21 # Normalisasi pixel sebelum prediksi 22 pixel_center_scaled = scaler.transform([pixel_center])</pre> SVM: Cool Grey 

7. Kesimpulan

Kesimpulan

1. Implementasi SVM dalam Deteksi Warna

- Algoritma **Support Vector Machine (SVM)** berhasil digunakan untuk mengklasifikasikan warna berdasarkan nilai piksel yang diambil dari gambar kamera.
- Model SVM mampu membedakan warna dengan baik jika dataset pelatihan memiliki distribusi warna yang cukup beragam.

2. Pendeteksian Multi-Warna Secara Simultan

- Dengan mengambil sampel dari **beberapa titik dalam frame**, sistem mampu mendeteksi lebih dari satu warna secara bersamaan.
- Hal ini meningkatkan kemampuan sistem dalam mengenali variasi warna dalam suatu gambar secara real-time.

8. Saran

1. Meningkatkan Kualitas Dataset

- Gunakan **lebih banyak sampel warna** dengan berbagai pencahayaan untuk meningkatkan akurasi model.
- Lakukan **augmentasi data** dengan variasi warna yang lebih luas untuk melatih model agar lebih robust.

2. Eksplorasi Algoritma Lain

- Jika SVM belum optimal, dapat mencoba **Random Forest**, **K-Nearest Neighbors (KNN)**, atau **CNN (Convolutional Neural Network)** untuk klasifikasi warna yang lebih kompleks.

9. Daftar Pustaka

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Vapnik, V. (1998). *Statistical Learning Theory*. Wiley.

Cortes, C., & Vapnik, V. (1995). "Support-Vector Networks." *Machine Learning*, **20**, 273–297
